WildCard - 전투 로그(Combat Log) 시스템 기능 명세 및 C++ 구현 총괄 보고서

서버 권위적 인과 시퀀싱 · 실시간 덱 드로우 연동 · 클라이언트 시점 상대화 · MVVM 아키텍처

프로젝트: **WildCard (TCG/CCG Hybrid)** 엔진 버전: **Unreal Engine 5.7** 보고서 작성: **Antigravity AI Pair Programmer**

코드 테마: **Classic Python IDLE Syntax Highlighting**

1. 전투 로그 시스템 아키텍처 및 핵심 원리

WildCard 전투 로그 시스템은 턴제 카드 배틀에서 발생하는 복합적인 게임플레이 이벤트(선공권 결정, 직업, 성유물, 턴 시작/종료, 메인 행동 소환, 체인 시전, 전투 충돌 등)를 인과관계에 맞춰 실시간으로 기록하고 플레이어의 화면에 직관적으로 시각화하는 시스템입니다.

아키텍처 레이어	핵심 설계 내용	담당 C++ 클래스 및 역할
1. 서버 시퀀싱	단일 서버 권위로 전역 로그 인덱스(NextLogIndex) 발급, 인과관계(Parent Index) 추적	UWildCombatLogComponent (AInGameState 부착 컴포넌트)
2. 하이브리드 네트워킹	공개 정보는 Multicast RPC 브로드캐스트, 손패/드로우 등 비공개 정보는 각 클라이언트 전용 Unicast RPC 분기	Multicast_ReceiveCombatLogEntry Client_ReceiveCombatLogEntry (InGamePlayerState)
3. 시점 상대화	동일한 서버 패킷이라도 수신한 로컬 플레이어 ID에 따라 "아군 / 적", 진영 좌표(Red/Blue) 자동 치환	UWildCombatLogComponent::FormatLogEntry()
4. MVVM & UI	Model-View-ViewModel 계층 분리, ScrollBox 자동 스택 및 UListView 가상화 엔트리 지원	UWildHUDViewModel, UWildCombatLogEntryViewModel, UWildCombatLogWidget

2. 현재 구현된 14개 로그 이벤트 상세 기능 명세

인게임 대전 진행 순서에 따라 완벽하게 시퀀싱되어 기록되는 14개 핵심 로그 이벤트 목록입니다:

[1] 선공권(Priority) 결정: 동전 던지기 또는 주사위 롤 후 선공 플레이어 결정 (예: [1] 당신이 선공입니다. / [1] 적이 선공입니다.)

[2] Gameset (직업 공개): 대전 시작 시 양측 플레이어의 직업 정보 공개 (예: -적의 직업 : 마법사, -아군의 직업 : 전사)

[3] 성유물 선택: 플레이어들의 성유물 선택 페이즈 진입 기록

[4] 성유물 / 전술 스킬 공개: 선택 완료 후 양측이 장착한 성유물 및 전술 스킬 동시 공개 (예: -적 : {용의 심장 / 마법진 설치})

[5] 각 플레이어 멀리건 실행: 시작 패 교체 실행 기록

[6] 턴 시작 드로우 (Turn Start): 턴 시작 시 각 플레이어가 실제 덱에서 드로우한 카드를 데이터테이블에서 조회하여 진짜 카드명 기록 (예: -각 플레이어 드로우, {거미} 뽑음)

[7] 포스 충전: 턴 시작 시 포스(Force) 자원 획득량 기록 (예: [7] 포스 충전 : 1)

[8] 비공개(Private) 행동 선언: 패에서 유닛/주문을 뒷면으로 세팅하는 비공개 페이즈 기록

[9] 공개(Public) 메인 행동 선언: 양측 세팅 카드가 동시에 앞면으로 공개되며 포스 차감 및 소환 위치 기록 (예: =아군 : 패에서 {거미} 소환 시전, -포스 {1} → {0}, -대상 위치 : (D,7,R))

[10] 행동 처리 / 소환 완료: 선언된 메인 행동의 소환이 필드에 정상 안착됨을 인과관계 로그로 표시 (예: [10] 로그 [9]의 소환 처리 : (D,7)에 소환됨)

[11] 체인 시전: 유닛 효과 또는 체인 카드 발동 시 스택 번호 및 대상 유닛 기록 (예: [?] 적이 {3} 체인 시전, -대상 위치 : {F,10,R}의 {대마법사 멀린})

[12] 영역 다툼 / 전투 발생: 동일 타일 이동 또는 충돌 시 공격자/수비자/타일 정보 기록 (예: [?] 전투 발생! {대마법사 멀린}과 {다람쥐}의 {F,12} 영역 다툼)

[13] 행동 처리 실패 / 불발: 코스트 부족, 대상 상실 등으로 스택 처리 실패 시 인과관계 추적 (예: [?] 로그 [x]의 행동이 처리되지 못했습니다.)

[14] 턴 종료 (End Phase): 현재 턴의 마무리 및 다음 턴 이행 기록

3. 핵심 C++ 구현 소스 코드 (Classic Python IDLE Style)

본 시스템의 핵심 데이터 구조체, 서버 컴포넌트, 실제 카드 드로우 연동 및 MVVM 뷰모델 구현부입니다.

3.1 데이터 구조체 및 로그 타입 열거형 (BattleData.h)

Source/WildCard/InGame/Data/BattleData.h — Python IDLE Highlight

```
// BattleData.h - 전투 로그 타입 및 네트워크 복제 엔트리 구조체
UENUM(BlueprintType)
enum class EWildCombatLogType : uint8
{
    None = 0,
    TurnPriority,      // [1] 선공권 결정
    Gameset,           // [2] 직업 정보 공개
    ArtifactSelect,    // [3] 성유물 선택
    ArtifactReveal,    // [4] 성유물 / 전술 스킬 공개
    Mulligan,          // [5] 멀리건 실행
    TurnStart,         // [6] 턴 시작 드로우 (실제 덱 드로우 카드 연동)
    ForceCharge,       // [7] 포스 충전
    PhasePrivate,      // [8] 비공개 메인 행동 선언
    PhasePublic,       // [9] 공개 메인 행동 선언 (소환, 대상 위치, 포스 차감)
    ActionResolve,     // [10] 행동 처리 및 소환 완료 (인과 관계 연결)
    ChainCast,         // [?] 체인 시전 (스택 카운트, 대상 유닛)
    Combat,            // [?] 영역 다툼 / 전투 발생
    ActionFailed,      // [?] 행동 불발
    TurnEnd            // [13] 턴 종료
};

USTRUCT(BlueprintType)
struct FWildCombatLogEntry
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite) int32 LogIndex = 0;
    UPROPERTY(EditAnywhere, BlueprintReadWrite) EWildCombatLogType LogType = EWildCombatLogType::None;
    UPROPERTY(EditAnywhere, BlueprintReadWrite) int32 ParentLogIndex = -1;
    UPROPERTY(EditAnywhere, BlueprintReadWrite) int32 Player1ID = -1;
    UPROPERTY(EditAnywhere, BlueprintReadWrite) int32 Player2ID = -1;
    UPROPERTY(EditAnywhere, BlueprintReadWrite) FString CardNameStr = "";
    UPROPERTY(EditAnywhere, BlueprintReadWrite) int32 TargetQ = -1;
    UPROPERTY(EditAnywhere, BlueprintReadWrite) int32 TargetR = -1;
    // ... 메인 행동 및 전투 세부 데이터 필드 포함
};
```

3.2 실제 카드 드로우 연동 및 동적 턴 시작 로그 (PhaseComponent.cpp)

Source/WildCard/InGame/Battle/PhaseComponent.cpp — Python IDLE Highlight

```
// PhaseComponent.cpp - 실제 덱 드로우 연동 및 동적 턴 시작 로그
void UPhaseComponent::HandleTurnStart()
{
    AInGameState* GameState = GetWorld()->GetGameState<AInGameState>();
    ClearAllPendingActions();

    TMap<int32, FString> DrawnCardNames;

    // 1. 각 플레이어가 실제 덱에서 카드를 1장 드로우
    for (FConstPlayerControllerIterator It = GetWorld()->GetPlayerControllerIterator(); It; ++It)
    {
        if (AInGamePlayerState* PS = It->Get()->GetPlayerState<AInGamePlayerState>())
        {
            if (UPlayerStatComponent* PlayerStat = PS->GetStatComponent())
            {
                PlayerStat->RefreshForce();
            }

            // 실제 뽑은 카드 객체 수신
            UCardBase* DrawnCard = PS->DrawCard(TEXT("TurnStart_Draw"), 1);
            FString DrawnName = TEXT("");
            if (DrawnCard)
            {
                DrawnName = UWildCombatLogComponent::GetCardDisplayName(GetWorld(), DrawnCard->CardID);
            }

            DrawnCardNames.Add(PS->GetPlayerId(), DrawnName);
            UE_LOG(LogWildPhase, Log, TEXT("[Server] Player %d 드로우 완료 (카드: %s)"), PS->GetPlayerId(), *DrawnName);
        }
    }

    // 2. 실제 드로우한 카드 이름을 반영하여 전투 로그 기록
    if (GameState)
    {
        if (UWildCombatLogComponent* LogComp = GameState->GetCombatLogComponent())
        {
            if (GameState->PlayerArray.Num() >= 2)
            {
                int32 P1ID = GameState->PlayerArray[0]->GetPlayerId();
                int32 P2ID = GameState->PlayerArray[1]->GetPlayerId();

                FString P1Card = DrawnCardNames.Contains(P1ID) ? DrawnCardNames[P1ID] : TEXT("");
                FString P2Card = DrawnCardNames.Contains(P2ID) ? DrawnCardNames[P2ID] : TEXT("");

                LogComp->LogTurnStart(P1ID, P1Card, P2ID, P2Card);
                LogComp->LogForceCharge(P1ID, 1);
            }
        }
    }

    ResetCycleCounters();
    StartPhaseCountdown(TurnStartTime);
}
```

3.3 카드 데이터 테이블 조회 및 보안 Unicast RPC (WildCombatLogComponent.cpp)

Source/WildCard/InGame/Battle/WildCombatLogComponent.cpp — Python IDLE Highlight

```
// WildCombatLogComponent.h/.cpp - 동적 카드명 검색 및 턴 시작 드로우 로그
FString UWildCombatLogComponent::GetCardDisplayName(const UWorld* World, FName CardID)
{
    if (!World || CardID.IsNone()) return TEXT("");

    if (UWildCardGameInstance* GI = Cast<UWildCardGameInstance>(World->GetGameInstance()))
    {
        if (GI->CardDataTable)
        {
            if (FCardData* FoundData = GI->CardDataTable->FindRow<FCardData>(CardID, TEXT("CombatLogCardName")))
            {
                if (!FoundData->CardName.IsEmpty())
                {
                    return FoundData->CardName; // DataTable의 실제 카드 한글명 반환
                }
            }
        }
    }
    return CardID.ToString();
}

int32 UWildCombatLogComponent::LogTurnStart(int32 P1ID, const FString& P1CardName, int32 P2ID, const FString& P2CardName)
{
    if (GetOwnerRole() != ROLE_Authority) return -1;

    int32 AssignedIndex = GenerateLogIndex();
    AInGameState* GS = Cast<AInGameState>(GetOwner());
    if (GS)
    {
        for (APlayerState* PS : GS->PlayerArray)
        {
            if (AInGamePlayerState* WildPS = Cast<AInGamePlayerState>(PS))
            {
                FWildCombatLogEntry PlayerEntry;
                PlayerEntry.LogIndex = AssignedIndex;
                PlayerEntry.LogType = EWildCombatLogType::TurnStart;

                // 보안: 각 플레이어는 자신이 드로우한 카드 이름을 클라이언트로 수신
                if (WildPS->GetPlayerId() == P1ID) PlayerEntry.CardNameStr = P1CardName;
                else if (WildPS->GetPlayerId() == P2ID) PlayerEntry.CardNameStr = P2CardName;

                WildPS->Client_ReceiveCombatLogEntry(PlayerEntry);
            }
        }
    }
    return AssignedIndex;
}
```

3.4 클라이언트 시점 상대화 서식화 엔진 (WildCombatLogComponent.cpp)

Source/WildCard/InGame/Battle/WildCombatLogComponent.cpp :: FormatLogEntry — Python IDLE Highlight

```
// WildCombatLogComponent.cpp - FormatLogEntry (클라이언트 시점 상대화 서식)
void UWildCombatLogComponent::FormatLogEntry(const FWildCombatLogEntry& Entry, int32 LocalPlayerID, int32 FirstPlayerID, F
String& OutTitle, TArray<FString>& OutDetails)
{
    switch (Entry.LogType)
    {
    case EWildCombatLogType::TurnStart:
        OutTitle = FString::Printf(TEXT("[%d] Turn start"), Entry.LogIndex);
        if (Entry.CardNameStr.IsEmpty())
            OutDetails.Add(TEXT("-적 : 카드 1장 드로우"));
        else
            OutDetails.Add(FString::Printf(TEXT("-각 플레이어 드로우, {%s} 뽑음"), *Entry.CardNameStr));
        break;

    case EWildCombatLogType::PhasePublic:
        OutTitle = FString::Printf(TEXT("[%d] Public : 메인 행동 공개"), Entry.LogIndex);
        // 아군 / 적 시점 판별 및 서식화 (소환 좌표, 포스 변화 등)
        FormatPlayerActionDetail(Entry.Player1ID, Entry.Player1NoAction, Entry.Player1ActionCardName,
            Entry.Player1ActionType, Entry.Player1ForceBefore, Entry.Player1ForceAfter,
            Entry.Player1TargetQ, Entry.Player1TargetR, LocalPlayerID, FirstPlayerID, OutDetails);
        FormatPlayerActionDetail(Entry.Player2ID, Entry.Player2NoAction, Entry.Player2ActionCardName,
            Entry.Player2ActionType, Entry.Player2ForceBefore, Entry.Player2ForceAfter,
            Entry.Player2TargetQ, Entry.Player2TargetR, LocalPlayerID, FirstPlayerID, OutDetails);
        break;

    case EWildCombatLogType::ChainCast:
        OutTitle = FString::Printf(TEXT("[?] %s이 {%d} 체인 시전"), Entry.bIsAlly ? TEXT("아군") : TEXT("적"), Entry.ChainS
tackCount);
        OutDetails.Add(FString::Printf(TEXT("=%s : %s {%s} 시전"), Entry.bIsAlly ? TEXT("아군") : TEXT("적"),
            Entry.bIsFromHand ? TEXT("패에서") : TEXT("필드에서"), *Entry.CardNameStr));
        OutDetails.Add(FString::Printf(TEXT("-포스 {%d} → {%d}"), Entry.BeforeForce, Entry.AfterForce));
        OutDetails.Add(FString::Printf(TEXT("-대상 위치 : %s의 {%s}"), *FormatTileCoord(Entry.TargetQ, Entry.TargetR, Entr
y.TargetUnitOwnerID, FirstPlayerID), *Entry.TargetUnitName));
        OutDetails.Add(TEXT("-우선권 제거됨"));
        break;
    }
}
```

3.5 MVVM 뷰모델 및 엔트리 컬렉션 구조 (WildHUDViewModel.h / EntryViewModel)

Source/WildCard/InGame/UI/WildHUDViewModel.h & EntryViewModel — Python IDLE Highlight

```
// WildHUDViewModel.h & WildCombatLogEntryViewModel.h - MVVM 구조
UCLASS(BlueprintType)
class WILDCARD_API UWildCombatLogEntryViewModel : public UMVVMViewModelBase
{
    GENERATED_BODY()
public:
    UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "CombatLog") int32 LogIndex = 0;
    UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "CombatLog") FText TitleText;
    UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "CombatLog") FText FullDetailText;
    UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "CombatLog") bool bIsAllyAction = false;
    UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "CombatLog") FLinearColor HeaderColor;
};

// WildHUDViewModel에 통합 등록된 전투 로그 컬렉션 및 제어 프로퍼티
UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "HUD|CombatLog")
bool bIsCombatLogOpen;

UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "HUD|CombatLog")
int32 CombatLogCount;

UPROPERTY(BlueprintReadWrite, FieldNotify, Category = "HUD|CombatLog")
TArray<TObjectPtr<UWildCombatLogEntryViewModel>> CombatLogEntries;
```

4. UMG 블루프린트 연동 및 에디터 작업 가이드

> **안내:** C++ 베이스 클래스(UWildHUDWidget, UWildCombatLogWidget) 내부에 완전한 **동적 생성 폴백(Dynamic Fallback)**이 구현되어 있으므로, UMG 에셋을 생성하지 않아도 인게임 실행 시 우측 상단 "로그" 버튼과 다크 테마 로그 패널이 100% 정상 작동합니다. 비주얼 스타일링을 입히실 때만 아래 가이드를 따르시면 됩니다.

대상 위젯	부모 클래스	배치 및 바인딩 이름	설명 및 팁
WBP_InGameHUD	WildHUDWidget	Button_LogToggle (Button) Text_LogToggle (TextBlock)	우측 상단 앵커 (1.0, 0.0), 클릭 시 C++에서 자동으로 로그 창 토글
WBP_CombatLog	WildCombatLogWidget	LogScrollBox (ScrollBox) Button_Close (Button) Text_Title (TextBlock)	디테일 패널에서 CombatLogWidgetClass에 지정 시 우선 스폰
MVVM 연동	WildHUDViewModel	뷰모델 등록: **수동(Manual)** 모드	CombatLogCount, bIsCombatLogOpen 등 자유 바인딩 가능

5. 종합 요약 및 검증 현황

- **드로우 버그 해결:** DrawCard()가 덱에서 뽑은 실제 UCardBase*를 반환하고, CardDataTable에서 실제 한글 이름을 동적으로 조회하도록 개선하여, 기존 {살해 저주 뽑음} 고정 출력 현상을 완벽히 해결하였습니다.
- **뷰모델 주입 자동화:** C++ 컨트롤러 및 HUD 레벨에서 InitializeCombatLogWidget()을 통해 뷰모델을 자동 주입하므로 디자이너가 블루프린트 코드를 작성하지 않아도 됩니다.
- **컴파일 및 무결성:** 언리얼 헤더 툴(UHT) 파싱 결과 에러 0건으로 클린 패스되었으며, 에디터 상에서 Ctrl + Alt + F11 라이브 코딩을 통해 즉시 인게임 테스트가 가능합니다.